| | |
|---|---|
| **Status** | Finished |
| **Started** | Wednesday, 21 October 2026, 6:29 PM |
| **Completed** | Wednesday, 21 October 2026, 6:45 PM |
| **Duration** | 15 mins 50 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

Question **1**

Correct

Mark 1.00 out of 1.00

The program must accept **N** integers and an integer **K** as the input. The program must print every K integers in descending order as the output.

**Note**: If **N % K != 0**, then sort the final N%K integers in descending order.

**Boundary Condition(s):**

1 <= N <= 10^4
-99999 <= Array Element Value <= 99999

**Input Format:**

The first line contains the values of N and K separated by a space.
The second line contains N integers separated by space(s).

**Output Format:**

The first line contains N integers.

**Example Input/Output 1:**

Input:

7 3
48 541 23 68 13 41 6

Output:

541 48 23 68 41 13 6

Explanation:

The first three integers are 48 541 23, after sorting in descending order the integers are **541 48 23**.
The second three integers are 68 13 41, after sorting in descending order the integers are **68 41 13**.
The last integer is **6**.
The integers are **541 48 23 68 41 13 6**
Hence the output is **541 48 23 68 41 13 6**.

**Answer:** (penalty regime: 0 %)

```
import sys

def main():

    input_data = sys.stdin.read().split()
    if not input_data:
        return
```

```
        return


    N = int(input_data[0])
    K = int(input_data[1])


    arr = [int(x) for x in input_data[2:2+N]]

    result = []


    for i in range(0, N, K):
        chunk = arr[i:i+K]

        chunk.sort(reverse=True)
        result.extend(chunk)

    print(*(result))

if __name__ == '__main__':
    main()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 7 3<br>48 541 23 68 13 41 6 | 541 48 23 68 41 13 6 | 541 48 23 68 41 13 6 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given a matrix mat where every row is sorted in **strictly increasing** order, return the **smallest common element** in all rows.

If there is no common element, return -1.

**Example 1:**

**Input:**

4 5

1 2 3 4 5

2 4 5 8 10

3 5 7 9 11

1 3 5 7 9

**Output:**

5

**Constraints:**

- 1 <= mat.length, mat[i].length <= 500
- 1 <= mat[i][j] <= 10^4
- mat[i] is sorted in strictly increasing order.

**Answer:** (penalty regime: 0 %)

```
import sys

def main():

    input_data = sys.stdin.read().split()
    if not input_data:
        return


    R = int(input_data[0])
    C = int(input_data[1])


    count = [0] * 10001

    idx = 2
    for r in range(R):
        for c in range(C):
            val = int(input_data[idx])
            count[val] += 1
            idx += 1


    for num in range(1, 10001):
        if count[num] == R:
            print(num)
```

```
            print(num)
            return


    print(-1)

if __name__ == '__main__':
    main()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 4 5<br>1 2 3 4 5<br>2 4 5 8 10<br>3 5 7 9 11<br>1 3 5 7 9 | 5 | 5 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the $p^{th}$ element of the list, sorted ascending. If there is no $p^{th}$ element, return 0.

**Example**

n = 20

p = 3

The factors of 20 in ascending order are {1, 2, 4, 5, 10, 20}. Using 1-based indexing, if p = 3, then 4 is returned. If p > 6, 0 would be returned.

**Constraints**

$1 \leq n \leq 10^{15}$

$1 \leq p \leq 10^{9}$

The first line contains an integer n, the number to factor.

The second line contains an integer p, the 1-based index of the factor to return.

**Sample Case 0**

**Sample Input 0**

10

3

**Sample Output 0**

5

**Explanation 0**

Factoring n = 10 results in {1, 2, 5, 10}. Return the p = $3^{rd}$ factor, 5, as the answer.

**Sample Case 1**

**Sample Input 1**

10

5

**Sample Output 1**

0

**Explanation 1**

Factoring n = 10 results in {1, 2, 5, 10}. There are only 4 factors and p = 5, therefore 0 is returned as the answer.

**Sample Case 2**

**Sample Input 2**

1

1

**Sample Output 2**

1

**Explanation 2**

Factoring n = 1 results in {1}. The p = 1st factor of 1 is returned as the answer.

**For example:**

| Input | Result |
|---|---|
| 10<br>3 | 5 |
| 10<br>5 | 0 |
| 1<br>1 | 1 |

**Answer:** (penalty regime: 0 %)

```
import sys
import math

def main():

    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    p = int(input_data[1])

    low_factors = []
    high_factors = []


    i = 1
    while i * i <= n:
        if n % i == 0:
            low_factors.append(i)

            if i * i != n:
                high_factors.append(n // i)
        i += 1


    total_factors = len(low_factors) + len(high_factors)


    if p > total_factors:
        print(0)
        return


    if p <= len(low_factors):
        print(low_factors[p - 1])
    else:

        high_factors.reverse()
        print(high_factors[p - len(low_factors) - 1])

if __name__ == '__main__':
    main()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 10<br>3 | 5 | 5 | ✓ |
| ✓ | 10<br>5 | 0 | 0 | ✓ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 1<br>1 | 1 | 1 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Complete the program to count frequency of each element of an array. Frequency of a particular element will be printed once.

Sample Test Cases

Test Case 1

Input

7
23
45
23
56
45
23
40

Output

23 occurs 3 times
45 occurs 2 times
56 occurs 1 times
40 occurs 1 times

**Answer:** (penalty regime: 0 %)

```
import sys
from collections import Counter

def main():

    input_data = sys.stdin.read().split()
    if not input_data:
        return

    N = int(input_data[0])


    arr = [int(x) for x in input_data[1:1+N]]


    frequencies = Counter(arr)


    seen = set()
    for num in arr:
        if num not in seen:
            print(f"{num} occurs {frequencies[num]} times")
            seen.add(num)

if __name__ == '__main__':
    main()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 7<br>23<br>45<br>23<br>56<br>45<br>23<br>40 | 23 occurs 3 times<br>45 occurs 2 times<br>56 occurs 1 times<br>40 occurs 1 times | 23 occurs 3 times<br>45 occurs 2 times<br>56 occurs 1 times<br>40 occurs 1 times | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given two arrays of positive integers, for each element in the second array, find the total number of elements in the first array which are *less than or equal to* that element. Store the values determined in an array.

For example, if the first array is *[1, 2, 3]* and the second array is *[2, 4]*, then there are *2* elements in the first array *less than or equal to 2.* There are *3* elements in the first array which are *less than or equal to 4*. We can store these answers in an array, *answer = [2, 3]*.

**Program Description**

The program must return an array of *m* positive integers, one *for each maxes[i]* representing the total number of elements *nums[j]* satisfying *nums[j] ≤ maxes[i]* where *0 ≤ j < n* and *0 ≤ i < m,* in the given order.

The program has the following:

*nums[nums[0],...nums[n-1]]:* first array of positive integers
*maxes[maxes[0],...maxes[n-1]]:* second array of positive integers

**Constraints**

- $2 \le n, m \le 10^5$
- $1 \le nums[j] \le 10^9$, where $0 \le j < n$.
- $1 \le maxes[i] \le 10^9$, where $0 \le i < m$.

## Input Format For Custom Testing

Input from stdin will be processed as follows and passed to the program.

The first line contains an integer *n*, the number of elements in *nums*.
The next *n* lines each contain an integer describing *nums[j]* where *0 ≤ j < n.*
The next line contains an integer *m*, the number of elements in *maxes*.
The next *m* lines each contain an integer describing *maxes[i]* where *0 ≤ i < m.*

## Sample Case 0

**Sample Input 0**

```
4
1
4
2
4
2
3
5
```

**Sample Output 0**

```
2
4
```

**Explanation 0**

We are given *n = 4*, *nums = [1, 4, 2, 4]*, *m = 2*, and *maxes = [3, 5]*.

1. For *maxes[0] = 3*, we have *2* elements in *nums* (*nums[0] = 1* and *nums[2] = 2*) that are ≤ *maxes[0]*.
2. For *maxes[1] = 5*, we have *4* elements in *nums* (*nums[0] = 1*, *nums[1] = 4*, *nums[2] = 2*, and *nums[3] = 4*) that are ≤ *maxes[1]*.

Thus, the program returns the array *[2, 4]* as the answer.

Sample Case 1

**Sample Input 1**

```
5
2
10
5
4
8
4
3
1
7
8
```

**Sample Output 1**

```
1
0
3
4
```

**Explanation 1**

We are given, *n = 5*, *nums = [2, 10, 5, 4, 8]*, *m = 4*, and *maxes = [3, 1, 7, 8]*.

1. For *maxes[0] = 3*, we have *1* element in *nums* (*nums[0] = 2*) that is ≤ *maxes[0]*.
2. For *maxes[1] = 1*, there are *0* elements in *nums* that are ≤ *maxes[1]*.
3. For *maxes[2] = 7*, we have *3* elements in *nums* (*nums[0] = 2*, *nums[2] = 5*, and *nums[3] = 4*) that are ≤ *maxes[2]*.
4. For *maxes[3] = 8*, we have *4* elements in *nums* (*nums[0] = 2*, *nums[2] = 5*, *nums[3] = 4*, and *nums[4] = 8*) that are ≤ *maxes[3]*.

Thus, the program returns the array *[1, 0, 3, 4]* as the answer.

**Answer:** (penalty regime: 0 %)

```python
import sys
import bisect

def main():

    input_data = sys.stdin.read().split()
    if not input_data:
        return


    n = int(input_data[0])
    idx = 1
    nums = [int(x) for x in input_data[idx:idx+n]]
    idx += n


    m = int(input_data[idx])
    idx += 1
    maxes = [int(x) for x in input_data[idx:idx+m]]


    nums.sort()
```

```
23
24
25      for query in maxes:
26
27          count = bisect.bisect_right(nums, query)
28          print(count)
29
30  if __name__ == '__main__':
31      main()
32
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 4<br>1<br>4<br>2<br>4<br>2<br>3<br>5 | 2<br>4 | 2<br>4 | ✓ |
| ✓ | 5<br>2<br>10<br>5<br>4<br>8<br>4<br>3<br>1<br>7<br>8 | 1<br>0<br>3<br>4 | 1<br>0<br>3<br>4 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

An array is monotonic if it is either **monotone increasing** or **monotone decreasing**.
An array A is monotone increasing if for all i <= j, **A[i] <= A[j]**. An array A is monotone decreasing if for all i <= j, **A[i] >= A[j]**.

Write a program if n array is monotonic or not. Print "True" if is monotonic or "False" if it is not. Array can be monotone increasing or decreasing.

Input Format:

First line n-get number of elements

Next n Lines is the array of elements

Output Format:

True ,if array is monotone increasing or decreasing.

otherwise False is printed

Sample Input1

4

5

6

7

8

Sample Output1

True

Sample Input2

4

6

5

4

3

Sample Output2

True

Sample Input 3

4

6

7

8

7

Sample Output3

False

**For example:**

| Input | Result |
|---|---|
| 4<br>6<br>5<br>4<br>3 | True |

**Answer:** (penalty regime: 0 %)

```python
import sys

def main():
    # Read all input tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Parse array size
    n = int(input_data[0])

    # Parse the array elements
    arr = [int(x) for x in input_data[1:1+n]]

    # Flags to keep track of directional trends
    is_increasing = True
    is_decreasing = True

    # Check adjacent elements
    for i in range(len(arr) - 1):
        if arr[i] > arr[i + 1]:
            is_increasing = False
        if arr[i] < arr[i + 1]:
            is_decreasing = False

    # The array is monotonic if it remains entirely increasing OR decreasing
    if is_increasing or is_decreasing:
        print("True")
    else:
        print("False")

if __name__ == '__main__':
    main()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 4<br>6<br>5<br>4<br>3 | True | True | ✓ |
| ✓ | 4<br>3<br>5<br>7<br>4 | False | False | ✓ |
| ✓ | 4<br>1<br>6<br>9<br>2 | False | False | ✓ |
| ✓ | 4<br>9<br>6<br>4<br>2 | True | True | ✓ |
| ✓ | 3<br>2<br>1<br>4 | False | False | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[i] - A[j] = k, i != j.

Input Format

1. First line is number of test cases T. Following T lines contain:
2. N, followed by N integers of the array
3. The non-negative integer k

Output format

Print 1 if such a pair exists and 0 if it doesn't.

Example

Input

1

3

1

3

5

4

Output:

1

Input

1

3

1

3

5

99

Output

0

**For example:**

| Input | Result |
|---|---|
| 1<br>3<br>1<br>3<br>5<br>4 | 1 |
| 1<br>3<br>1<br>3<br>5<br>99 | 0 |

**Answer:** (penalty regime: 0 %)

```
import sys

def solve_test_case(A, N, k):
    # Two-pointer initialization
    j = 0
    i = 1

    while i < N and j < N:
        diff = A[i] - A[j]

        if diff == k and i != j:
            return 1
        elif diff < k:
            i += 1
        else:
            j += 1
            # Ensure pointers don't overlap as i must be != j
            if i == j:
                i += 1

    return 0

def main():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    ptr = 0
    # Number of test cases
    T = int(input_data[ptr])
    ptr += 1

    for _ in range(T):
        # Number of array elements
        N = int(input_data[ptr])
        ptr += 1

        # Parse the sorted array elements
        A = [int(x) for x in input_data[ptr:ptr+N]]
        ptr += N

        # Target difference k
        k = int(input_data[ptr])
        ptr += 1

        # Find if pair exists and print output
        print(solve_test_case(A, N, k))

if __name__ == '__main__':
    main()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 1<br>3<br>1<br>3<br>5<br>4 | 1 | 1 | ✓ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 1<br>3<br>1<br>3<br>5<br>99 | 0 | 0 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given an integer n, return an list of length n + 1 such that for each i (0 <= i <= n), ans[i] is the number of 1's in the binary representation of i.

Example:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
5 --> 101
```

Note: Complete the given function alone

**For example:**

| Test | Result |
|---|---|
| print(CountingBits(5)) | [0, 1, 1, 2, 1, 2] |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def CountingBits(n):
    # Initialize the list with zeros of length n + 1
    ans = [0] * (n + 1)

    # Fill the list using the dynamic programming transition
    for i in range(1, n + 1):
        # ans[i // 2] is equivalent to ans[i >> 1]
        # i % 2 is equivalent to i & 1
        ans[i] = ans[i // 2] + (i % 2)

    return ans

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | `print(CountingBits(2))` | `[0, 1, 1]` | `[0, 1, 1]` | ✓ |
| ✓ | `print(CountingBits(5))` | `[0, 1, 1, 2, 1, 2]` | `[0, 1, 1, 2, 1, 2]` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Assume you have an array of length ***n*** initialized with all **0**'s and are given ***k*** update operations.

Each operation is represented as a triplet: **[startIndex, endIndex, inc]** which increments each element of subarray **A[startIndex ... endIndex]** (startIndex and endIndex inclusive) with **inc**.

Return the modified array after all ***k*** operations were executed.

**Example:**

**Input:**

5

3

1 3 2

2 4 3

0 2 -2

**Output:**

-2 0 3 5 3

**Explanation:**

Initial state:

length = 5, updates = [[1,3,2],[2,4,3],[0,2,-2]]

[0,0,0,0,0]

After applying operation [1,3,2]:

[0,2,2,2,0]

After applying operation [2,4,3]:

[0,2,5,5,3]

After applying operation [0,2,-2]:

[-2,0,3,5,3]

**Answer:** (penalty regime: 0 %)

```
import sys

def main():
    # Read all inputs from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Parse length of the array (n) and number of updates (k)
    n = int(input_data[0])
    k = int(input_data[1])

```

```
    # Initialize the difference array with size n + 1 to handle bounds safely
    diff = [0] * (n + 1)

    # Read each of the k operations
    idx = 2
    for _ in range(k):
        start = int(input_data[idx])
        end = int(input_data[idx+1])
        inc = int(input_data[idx+2])
        idx += 3

        # Apply the update boundaries
        diff[start] += inc
        if end + 1 < n:
            diff[end + 1] -= inc

    # Reconstruct the modified array using running prefix sum
    current_val = 0
    result = []
    for i in range(n):
        current_val += diff[i]
        result.append(current_val)

    # Print the final array elements separated by a space
    print(*result)

if __name__ == '__main__':
    main()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 5<br>3<br>1 3 2<br>2 4 3<br>0 2 -2 | -2 0 3 5 3 | -2 0 3 5 3 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1

2

2

3

4

Output:

1 2 3 4

Example Input:

6

1

1

2

2

3

3

Output:

1 2 3

**For example:**

| Input | Result |
|---|---|
| 5<br>1<br>2<br>2<br>3<br>4 | 1 2 3 4 |
| 6<br>1<br>1<br>2<br>2<br>3<br>3 | 1 2 3 |

**Answer:** (penalty regime: 0 %)

```
n = int(input())
arr = []

while len(arr) < n:
    arr.extend(map(int, input().split()))

distinct = []
for num in arr:
    if num not in distinct:
        distinct.append(num)

print(*distinct)
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 5<br>1<br>2<br>2<br>3<br>4 | 1 2 3 4 | 1 2 3 4 | ✓ |
| ✓ | 6<br>1<br>1<br>2<br>2<br>3<br>3 | 1 2 3 | 1 2 3 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.